Lec02：面向对象编程四大特征及其解决的编程问题

The Four Characteristics of OOP and the Programming Problems They Solve

课程编号：22WHSE22006（2023版培养计划）

授课对象：23 级软件工程、服务科学与工程专业

主讲教师：辛国栋

Email：xinguodong@163.com

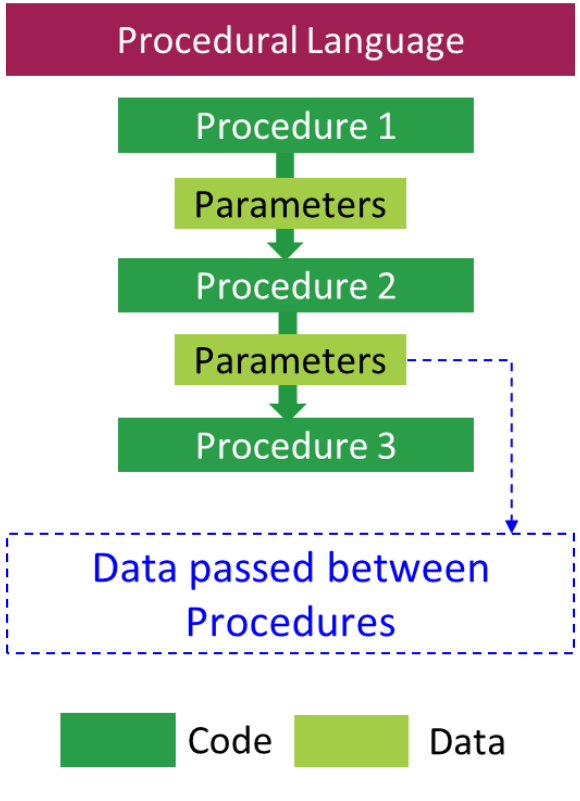


QQ 群：软件构造2025

群　号：311528424

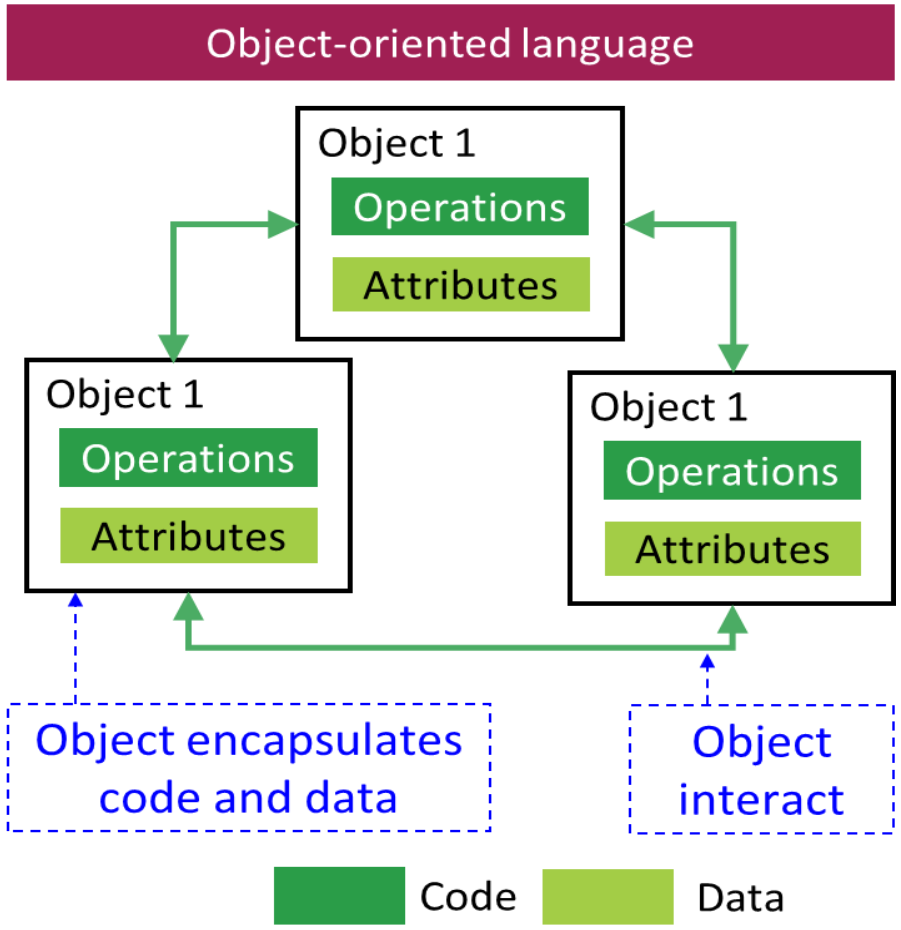
引言：面向过程的编程风格 Procedure Oriented

```c
struct User{
    char name[64];
    int age;
    char gender[16];
};
```




```c
struct User parse_to_user(char *text){
    // 将 text("小王 &28& 男") 解析成结构体 struct User
}
char *format_to_text(struct User user){
    // 将结构体 struct User 格式化成文本（" 小王\t28\t 男 "）
}
void format_user_file(char *origin_file_path, char *new_file_path){
    // open files...
    struct User users[1024]; // 假设最大 1024 个用户
    int count = 0;
    while (1)    { // read until the file is empty
        struct User user = parse_to_user(line);
        users[count++] = user;
    }
    sort_users_by_age(users);
    for (int i = 0; i < count; ++i)    {
        char *formatted_user_text = format_to_text(users[i]);
        // write to new file...
    }
    // close files...
}
```

引言：面向对象的编程风格 Object Oriented



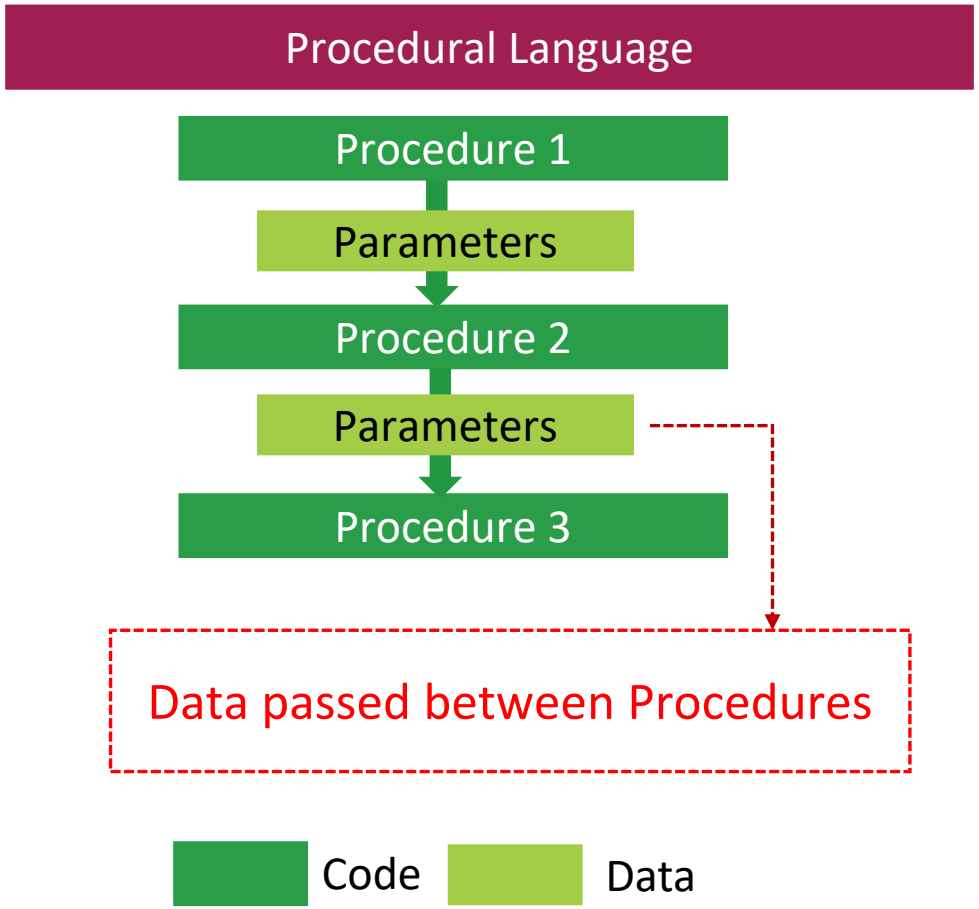
Object encapsulates code and data

```
public class User {
    private String name;
    private int age;
    private String gender;
    public User(String name, int age, String gender) { //代码略 }
    public User praseFrom(String userInfoText) { //代码略 }
    public String formatToText() { //代码略 }
}
```

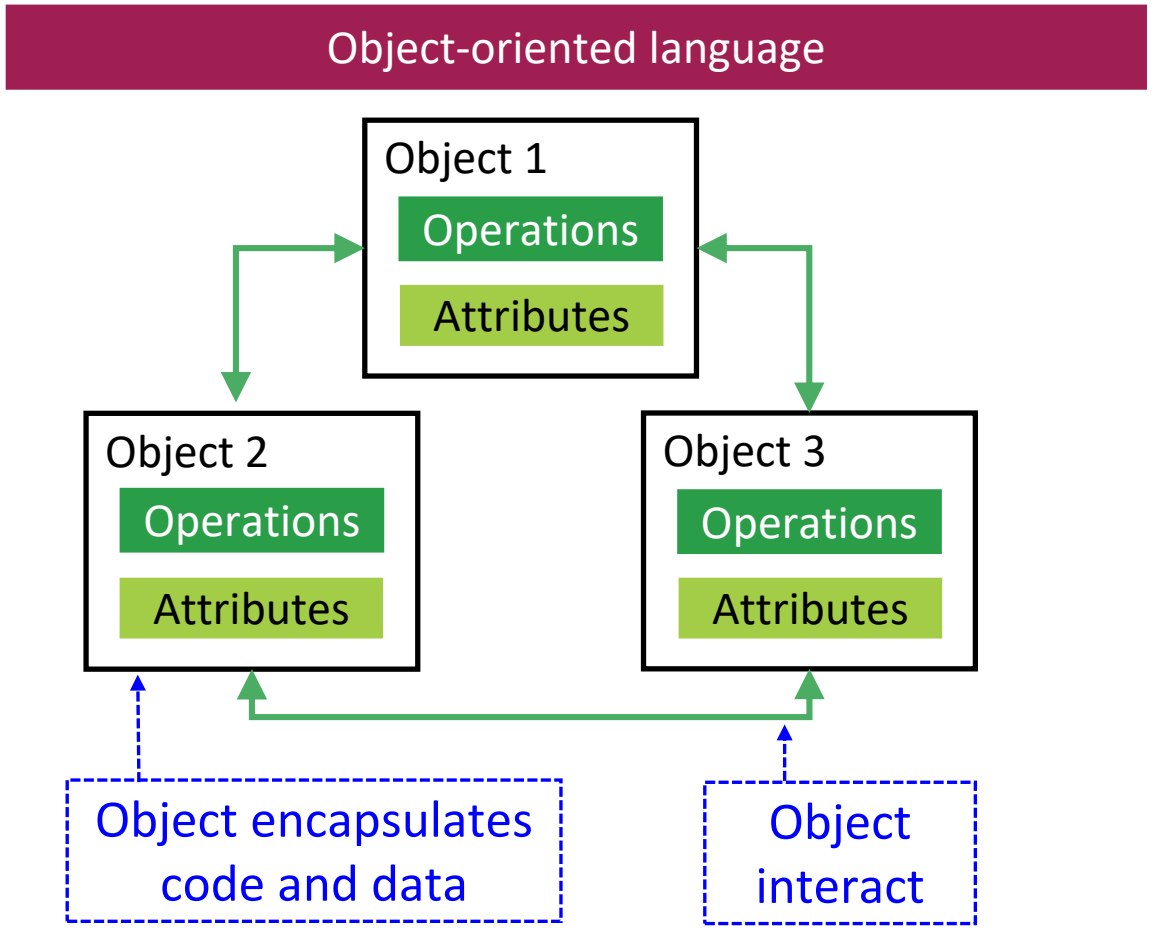
数据-Attributes

```
public class UserFileFormatter {
  public void format(String userFile, String formattedUserFile) {
        // Open files...
        List users = new ArrayList<>();
        while (1) { // read until file is empty
            // read from file into userText...
            User user = User.parseFrom(userText);
            users.add(user);        // object interact 消息传递
        }
        // close files...
  }
}
```

引言：面向过程与面向对象的编程思想对比

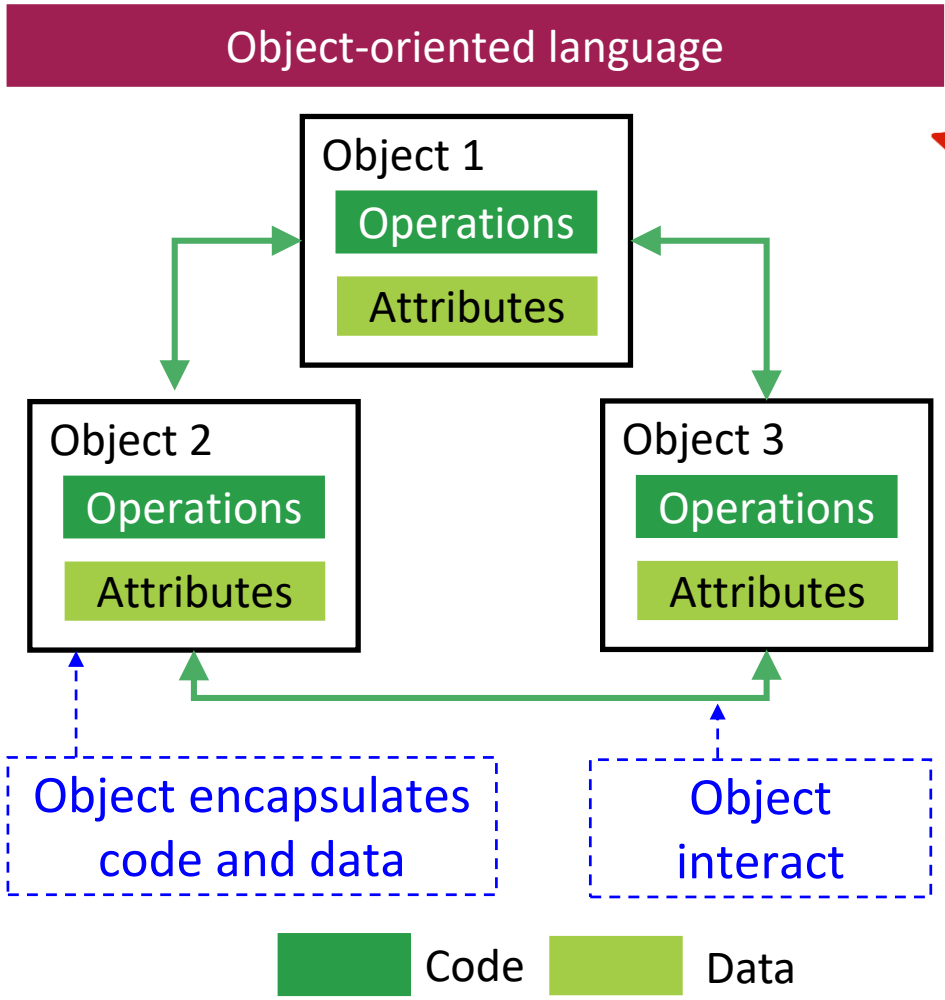


面向过程风格：方法和数据分离，通过方法调用交互



面向对象风格：方法和数据绑定在类中，通过消息传递进行交互

引言：面向对象的四大特征



对象中的数据（Attributes）：是对象的私有财产，只能通过对象提供的有限的必要操作（Operations）访问数据－需要数据隐藏或者数据访问保护

封装

对象中的操作：调用者只需要知道方法的签名（方法的名字，输入，返回值，注释）即可，不应该知道对象内部如何具体实现；此外，接口定义了所有实现类应该实现的方法，这也是一种抽象。

抽象

从人类认知的角度，真实世界中存在“is-a”的关系。面向对象亦如此。父类的财产可以被子类部分继承

继承

不同子类的对象，可以表现出不同的形态。

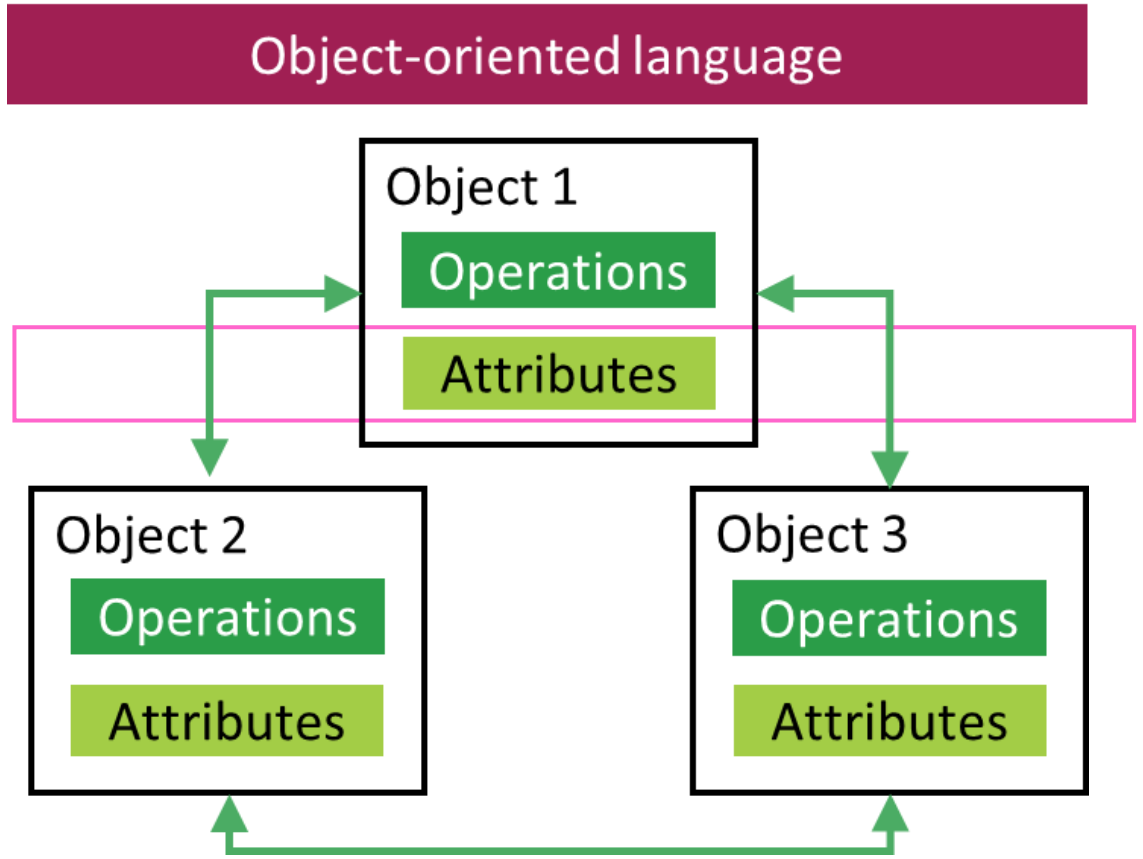
多态

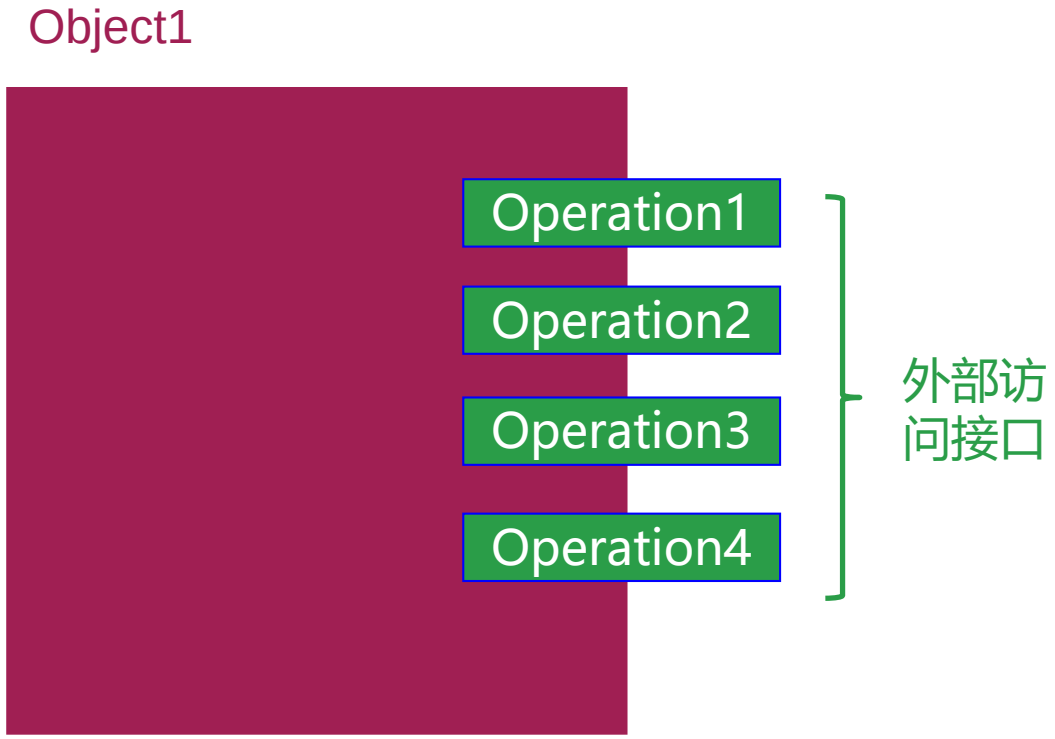
1. 封装

Encapsulation，美[ɪnˌkæpsjuˈleɪʃ(ə)n]

1.1 封装 Encapsulation 的定义

- 封装也叫作信息隐藏或者数据访问保护。类通过暴露有限的访问接口，授权外部访问内部数据





属性 Attributes，即对象的内部数据（也称为对象的状态），是一个黑盒，外部不可见

1.2 封装的实例说明

```
public class Wallet { //金融系统中一个简化版的虚拟钱包 。例如微信钱包，或者支付宝账户都属于此类型

    private String id;
    private long createTime;
    private BigDecimal balance; //表示钱包中的余额
    private long balanceLastModifiedTime; //上次钱包余额变更的时间。
    // ... 省略其他属性...，篇幅所限，省略实现代码，代码做了折叠
    public Wallet() {
        this.id = IdGenerator.getInstance().generate();
        this.createTime = System.currentTimeMillis();
        //为属性赋值,其他属性赋值的代码略
    }
    public String getId() { //代码略 }
    public long getCreateTime() { //代码略 }
    public BigDecimal getBalance() { //代码略  }
    public long getBalanceLastModifiedTime() { //代码略  }
    public void increaseBalance(BigDecimal increasedAmount) { //代码略 }
    public void decreaseBalance(BigDecimal decreasedAmount) { //代码略 }
}
```

1.2 封装的实例说明

```
public class Wallet {

    private String id;
    private long createTime;
    private BigDecimal balance;
    private long balanceLastModifiedTime;

    public Wallet() {
        this.id = IdGenerator.getInstance().generate();
        this.createTime = System.currentTimeMillis();
    }

    public String getId() { //代码略 }
    public long getCreateTime() { //代码略 }

    public BigDecimal getBalance() { //代码略  }
    public long getBalanceLastModifiedTime() { //代码略  }

    public void increaseBalance(BigDecimal increasedAmount) { //代码略 }
    public void decreaseBalance(BigDecimal decreasedAmount) { //代码略 }
```

根据封装特性，对钱包的四个属性的访问方式进行了限制，只允许通过下面的六个方法来访问或者修改钱包里的数据

1.2 封装的实例说明

```
public class Wallet {

    private String id;
    private long createTime;

    private BigDecimal balance;
    private long balanceLastModifiedTime;

     public Wallet() {
        this.id = IdGenerator.getInstance().generate();
        this.createTime = System.currentTimeMillis();
    }
    public String getId() { //代码略 }
    public long getCreateTime() { //代码略 }

    public BigDecimal getBalance() { //代码略  }
    public long getBalanceLastModifiedTime() { //代码略  }

    public void increaseBalance(BigDecimal increasedAmount) { //代码略 }
    public void decreaseBalance(BigDecimal decreasedAmount) { //代码略 }
```

id和createTime 两个属性在创建钱包时（new wallet()，通过构造函数注入）就已经确定，这两个属性不允许被修改

因此，id和createTime属性只有getter

备注：getXxx() 称为getter
setXxx() 称为setter

1.2 封装的实例说明

```
public class Wallet {

    private String id;
    private long createTime;

    private BigDecimal balance;
    private long balanceLastModifiedTime;

     public Wallet() {
        this.id = IdGenerator.getInstance().generate();
        this.createTime = System.currentTimeMillis();
    }
    public String getId() { //代码略 }
    public long getCreateTime() { //代码略 }

    public BigDecimal getBalance() { //代码略  }
    public long getBalanceLastModifiedTime() { //代码略  }

    public void increaseBalance(BigDecimal increasedAmount) { //代码略 }
    public void decreaseBalance(BigDecimal decreasedAmount) { //代码略 }
```

Balance属性可以读，但只能是增减，不能重新设置。否则无法保证数据的一致性和安全性。

因此，只保留这两个方法进行写操作，并没有setter

1.2 封装的实例说明

```
public class Wallet {

    private String id;
    private long createTime;

    private BigDecimal balance;
    private long balanceLastModifiedTime;

     public Wallet() {
        this.id = IdGenerator.getInstance().generate();
        this.createTime = System.currentTimeMillis();
    }
    public String getId() { //代码略 }
    public long getCreateTime() { //代码略 }

    public BigDecimal getBalance() { //代码略  }
    public long getBalanceLastModifiedTime() { //代码略  }

    public void increaseBalance(BigDecimal increasedAmount) { //代码略 }
    public void decreaseBalance(BigDecimal decreasedAmount) { //代码略 }
```

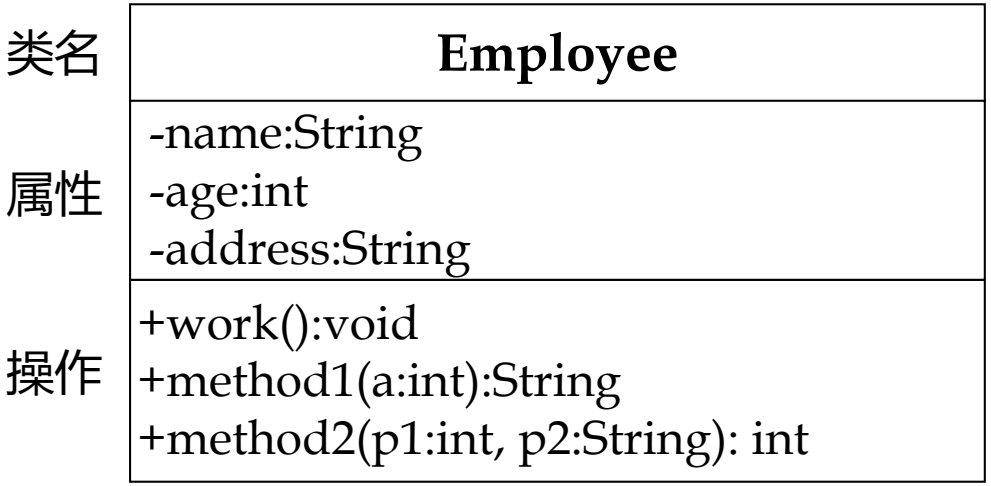
因为：只有在 balance 被修改的时候，此属性才会被修改。所以，把该属性的修改操作完全封装在下面两个方法中。
保证了 balance 和 balanceLastModifiedTime 两个数据的一致性。

此属性为何只有 getter 没有 setter?

1.3 封装这一特性需要语法机制才能实现-访问控制权限

无访问控制就谈不上封装。Java中的访问控制权限如下：

可见性修饰符		当前类	同包	其他包
+	public	√	√	√
#	protected	√	√	×
~	default	√	√	×
-	private	√	×	×




```
public class Employee {
    private String name;
    private int age;
    private String address;
    public void work(){  //…     }
    public String method1(int a){   //…   }
    public int method2(int p1,String p2){ //… }
}
```

1.4 封装解决的编程问题 – 数据隐藏，提高易用性

防止类中的数据被任意修改

如果类中属性不做限制，任何代码都能访问和修改：看起来灵活，但不可控。
此外，修改逻辑可能散落在代码的各个角落，势必影响代码的可读性和可维护性

仅仅通过有限的方法，暴露必要的操作，提高了类的易用性

将属性封装起来，暴露少许必要的方法给调用者使用，调用者就不必了解太多背后的业务细节，用错的概率就减少很多

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

```
public class ShoppingCart {       //模拟京东，淘宝，唯品会等电商网站类似的购物车
    private int itemsCount;       //商品数量
    private double totalPrice;    //商品总价
    private List<ShoppingCartItem> items = new ArrayList<>(); //商品列表

    public int getItemsCount() {  return this.itemsCount;    }
    public void setItemsCount(int itemsCount) { this.itemsCount = itemsCount;   }

    public double getTotalPrice() {  return this.totalPrice;    }
    public void setTotalPrice(double totalPrice) {   this.totalPrice = totalPrice;   }

    public List<ShoppingCartItem> getItems() {    return this.items;   }

    public void addItem(ShoppingCartItem item) {
        items.add(item);
        itemsCount++;
        totalPrice += item.getPrice();
    }    // ... 省略其他方法...
}
```

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

```
public class ShoppingCart {
    private int itemsCount;  //商品数量
    private double totalPrice; //商品总价
    private List<ShoppingCartItem> items = new ArrayList<>(); //商品列表

    public int getItemsCount() {  return this.itemsCount;    }
①   public void setItemsCount(int itemsCount) { this.itemsCount = itemsCount;   }

    public double getTotalPrice() {  return this.totalPrice;    }
②   public void setTotalPrice(double totalPrice) {   this.totalPrice = totalPrice;   }
```

①②代码出现的问题： 外部可以通过 setter 方法随意地修改 itemsCount totalPrice 两个属性的值，导致其跟 items 属性的值不一致

解决方案：去掉 itemsCount totalPrice 两个属性的 setter

```
}
```

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

```
public class ShoppingCart {
    private int itemsCount;  //商品数量
    private double totalPrice; //商品总价
    private List<ShoppingCartItem> items = new ArrayList<>(); //商品列表
```

③代码出现的问题：尽管 Items 属性未定义 setter，但该属性的 getter 方法，返回的是一个 List 集合容器。外部调用者在拿到这个容器之后，是可以操作容器内部数据的，即 Items 数据泄露。

```
③  public List<ShoppingCartItem> getItems() {    return this.items;    }
```

例如：在调用者中，编写如下代码：

```
ShoppingCart cart = new ShoppCart();
cart.getItems().clear(); // 清空购物车（直接利用 ArrayList 的 clear 方法，将数据清空）
```

会导致 itemsCount、totalPrice、items 三者数据不一致。

如何防止 items 数据泄露?

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

- 尝试解决 items 数据泄露问题：
  - 通过 Java 提供的 Collections.unmodifiableList() 方法，让 getter 方法返回一个不可被修改的 UnmodifiableList 集合容器，该容器是List的不可变包装类。
- 常见的常见的不可变包装 Unmodifiable Wrappers
  - public static <T> Collection<T> unmodifiableCollection(Collection<? extends T> c);
  - public static <T> Set<T> unmodifiableSet(Set<? extends T> s);
  - public static <T> List<T> unmodifiableList(List<? extends T> list);
  - public static <K,V> Map<K, V> unmodifiableMap(Map<? extends K, ? extends V> m);
  - public static <T> SortedSet<T> unmodifiableSortedSet(SortedSet<? extends T> s);
  - public static <K,V> SortedMap<K, V> unmodifiableSortedMap(SortedMap<K, ? extends V> m);

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

- 常用的 List.of、Set.of 和 Map.of 都是创建已有集合的不可变封装，例如：

```
List<String> a = List.of("lion", "tiger", "bear");
```

```
    public static void main(String[] args) {
        List<String> a = List.of("lion", "tiger", "bear");
        a.add("hahah");
        System.out.println(a.size());
    }
```

List.of 返回不可变的 list 封装unmodifiableList，因此向其添加数据时，必然发生异常

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.1\lib\id
Exception in thread "main" java.lang.UnsupportedOperationException Create breakpoint
    at java.base/java.util.ImmutableCollections.uoe(ImmutableCollections.java:142)
    at java.base/java.util.ImmutableCollections$AbstractImmutableCollection.add(ImmutableCollections.java:147)
    at Main.main(Main.java:22)
```

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

因此，我们解决尝试 items 数据泄露问题：通过 Java 提供的 Collections.unmodifiableList() 方法，让 getter 方法返回一个不可被修改的 UnmodifiableList 集合容器

```
public class ShoppingCart {
    // 省略其他代码
    public List<ShoppingCartItem> getItems() {
        // return this.items;
        return Collections.unmodifiableList(this.items);
    }
}
```

```
public class ShoppingCartTest {
    @Test
    public void testDataLeak(){
        ShoppingCart cart = new ShoppingCart();
        cart.getItems().clear();
    }
}
```



此时，cart.getItems() 返回的是 unmodifiableList 类型，不允许对 unmodifiableList 修改

能够看出，cart.getItems() 不允许修改了。
但是，是否真正解决了 数据泄露 的问题?

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

```java
public class ShoppingCart {
    // 省略其他代码
    public List<ShoppingCartItem> getItems() {
        // return this.items;
        return Collections.unmodifiableList(this.items);
    }
}
```

```java
public class ShoppingCartTest {
    @Test
    public void testDataLeak(){
        ShoppingCart cart = new ShoppingCart();
        cart.addItem(new ShoppingCartItem());
        List<ShoppingCartItem> items = cart.getItems();
        ShoppingCartItem item = items.get(0);
        item.setPrice(15.0); //设置列表存储的对象的价格
        System.out.println(item.getPrice());
    }
}
```

```
✓ 1 test passed  1 test total, 3 ms
D:\tools\jdk-17.0.12\bin\java.exe ...
15.0

Process finished with exit code 0
```

发现容器中存储的 ShoppingCartItem 依然可以修改，并没有解决数据的泄露问题。

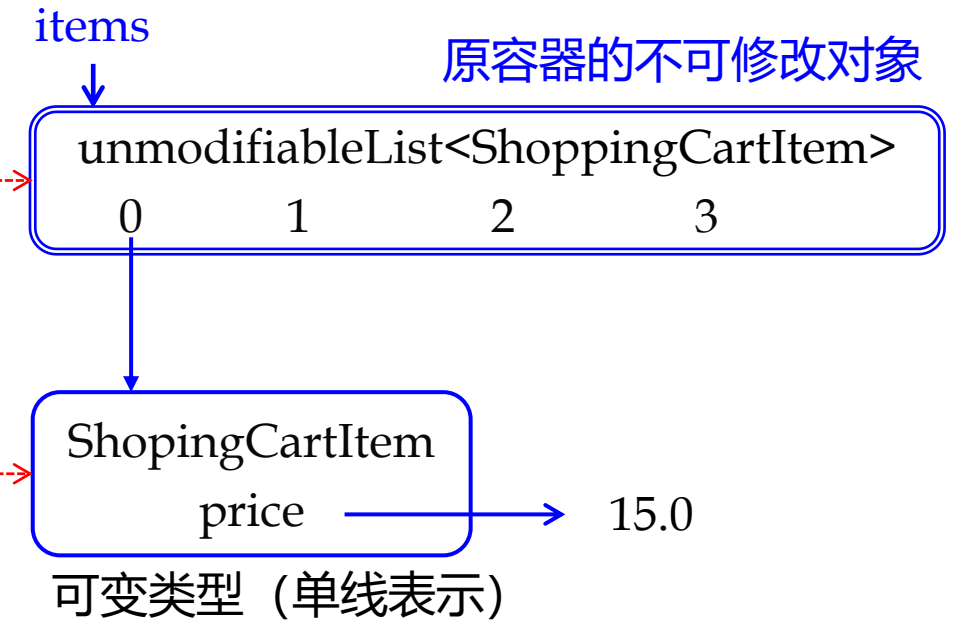
1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

```java
public class ShoppingCart {
    // 省略其他代码
    public List<ShoppingCartItem> getItems() {
        // return this.items;
        return Collections.unmodifiableList(this.items);
    }
}
```

```java
public class ShoppingCartTest {
    @Test
    public void testDataLeak(){
        ShoppingCart cart = new ShoppingCart();
        cart.addItem(new ShoppingCartItem());
        List<ShoppingCartItem> items = cart.getItems();

        ShoppingCartItem item = items.get(0);
        item.setPrice(15.0);
        System.out.println(item.getPrice());
    }
}
```

快照图（Snapshot Diagraph）。其中双线表示不可修改对象



items本身不可修改，但 items中存储的对象可以被修改，并没有解决数据泄露问题。

课程的后续内容将讨论解决方案

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

此外，在开发中，我们经常用 Lombok 库自动生成 setter/getter 等代码，以简化开发，提高代码可读性

```
public class Dept {
    private int deptno;
    private String deptname;

    public String getDeptname() {
        return deptname;
    }
    public void setDeptname(String deptname) {
        this.deptname = deptname;
    }
    public int getDeptno() {
        return deptno;
    }
    public void setDeptno(int deptno) {
        this.deptno = deptno;
    }
    // toString() 省略...
}
```

在Java Maven 项目的 pom.xml 文件中添加依赖：

```
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <version>1.18.36</version>
</dependency>
```

添加注解，左侧代码简化为如下：

```
@Data
public class Dept {
    private int deptno;
    private String deptname;
}
```

其中：@Data能够自动生成所有字段的 getter 和 setter方法，包括全字段的toString()方法、equals() 和 hashCode()方法，以及无参构造。

1.4 破坏封装典型示例 ① – 滥用 getter、setter 方法

- 为了避免封装泄露，并非所有的属性都需要加 getter/setter。Lombok 如何解决此问题?

方式一

```
// 去掉 @Data
public class Dept {
    @Getter  // 根据设计需要，在属性上标注 @Getter 或者 @Setter
    private int deptno;
    @Getter
    @Setter
    private String deptname;
}
```

方式二

```
@Data
public class Dept {
    @Setter(AccessLevel.NONE)// 对不需要的Getter/Setter，设置 AccessLevel.NONE
    private int deptno;
    private String deptname;
}
```

1.5 破坏封装典型示例 ② – 滥用全局变量和全局方法

- 在面向对象编程中，常见的全局变量有静态成员变量、常量等，常见的全局方法有静态方法。
  - 静态成员变量归属于类上的数据，被所有的实例化对象所共享，也相当于一定程度上的全局变量
  - 常量是一种非常常见的全局变量，比如一些代码中的配置参数，一般都设置为常量，放到一个 Constants 类中；
  - 静态方法一般用来操作静态变量或者外部数据。例如各种 Utils 类，里面的方法一般都会定义成静态方法，可以在不创建对象的情况下，直接拿来使用。静态方法将方法与数据分离，破坏了封装特性
- 以上情况出现的典型问题：方法与数据分离，破坏封装

1.5 破坏封装典型示例 ② – 滥用全局变量和全局方法

```
public class Constants {    //大的常量类，出现的典型问题：方法与数据分离

    public static final String MYSQL_ADDR_KEY = "mysql_addr";
    public static final String MYSQL_DB_NAME_KEY = "db_name";
    public static final String MYSQL_USERNAME_KEY = "mysql_username";
    public static final String MYSQL_PASSWORD_KEY = "mysql_password";

    public static final String REDIS_DEFAULT_ADDR = "192.168.7.2:7234";
    public static final int REDIS_DEFAULT_MAX_TOTAL = 50;
    public static final int REDIS_DEFAULT_MAX_IDLE = 50;
    public static final int REDIS_DEFAULT_MIN_IDLE = 20;
    public static final String REDIS_DEFAULT_KEY_PREFIX = "rt:";
    // ... 省略更多的常量定义...
}
```

多个人用同一个类，导致常量越来越多，类越来越大，容易重复，查起来也费劲，影响代码的可维护性；被多个类依赖，增加代码的编译时间；影响复用性：即便只用一个常量，也得把整个类引入

1.5 破坏封装典型示例 ② – 滥用全局变量和全局方法

解决方案1：写到配置文件（xml, properties, yaml, yml等），因为配置文件不需要编译，不影响编译时间。

```
public class Constants {

    public static final String MYSQL_ADDR_KEY = "mysql_addr";
    public static final String MYSQL_DB_NAME_KEY = "db_name";
    public static final String MYSQL_USERNAME_KEY = "mysql_username";
    public static final String MYSQL_PASSWORD_KEY = "mysql_password";
```

这部分和MySql数据库相关，抽出一个单独的配置类

解决方案2：拆分成多个配置类。MySQL 和 Redis 的配置分别做成配置类，一定程度减少编译时间

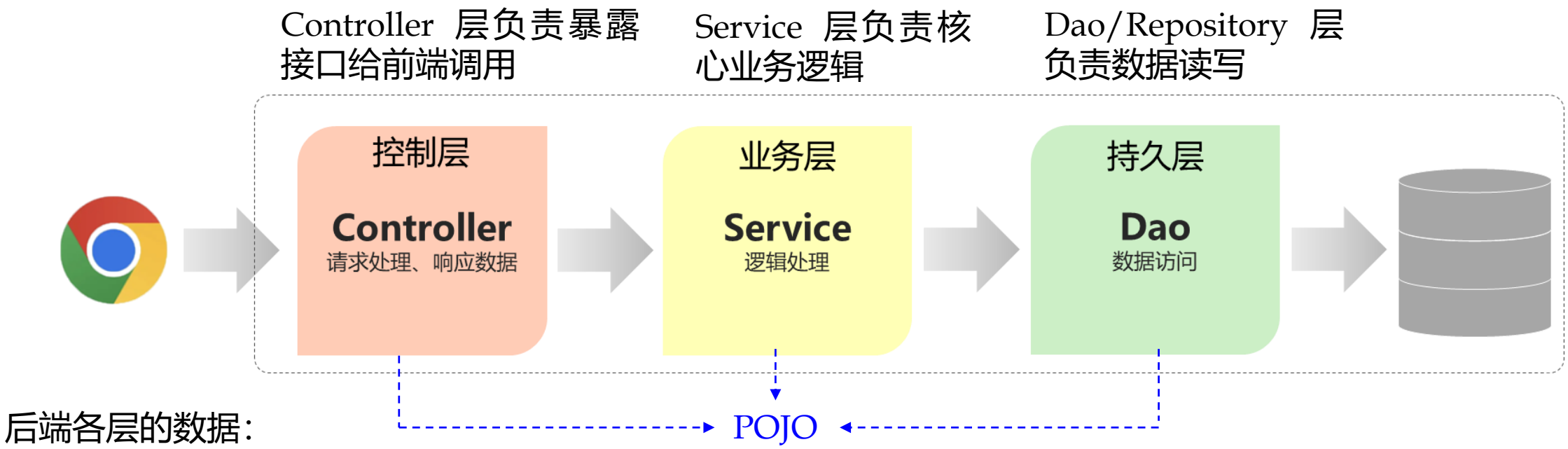
```
public static final String REDIS_DEFAULT_ADDR = "192.168.7.2:7234";
    public static final int REDIS_DEFAULT_MAX_TOTAL = 50;
    public static final int REDIS_DEFAULT_MAX_IDLE = 50;
    public static final int REDIS_DEFAULT_MIN_IDLE = 20;
    public static final String REDIS_DEFAULT_KEY_PREFIX = "rt:";
    // ... 省略更多的常量定义...

}
```

这部分和 REDIS 相关，抽出一个单独的配置类

1.6 破坏封装典型示例 ③ – 定义数据和方法分离的类

- 数据定义在一个类中，方法定义在另一个类中：例如，三层结构的 Web 后端开发



Controller、Service、Repository 各层均依赖于 POJO，POJO 负责各层的数据传输。根据POJO的功能不同，可以细化成 VO，BO，PO 等

1.6 破坏封装典型示例 ③ – 定义数据和方法分离的类

- 对持续迭代的团队协作项目，业务复杂，人员协同性要求高，要求各层实现自己的实体类



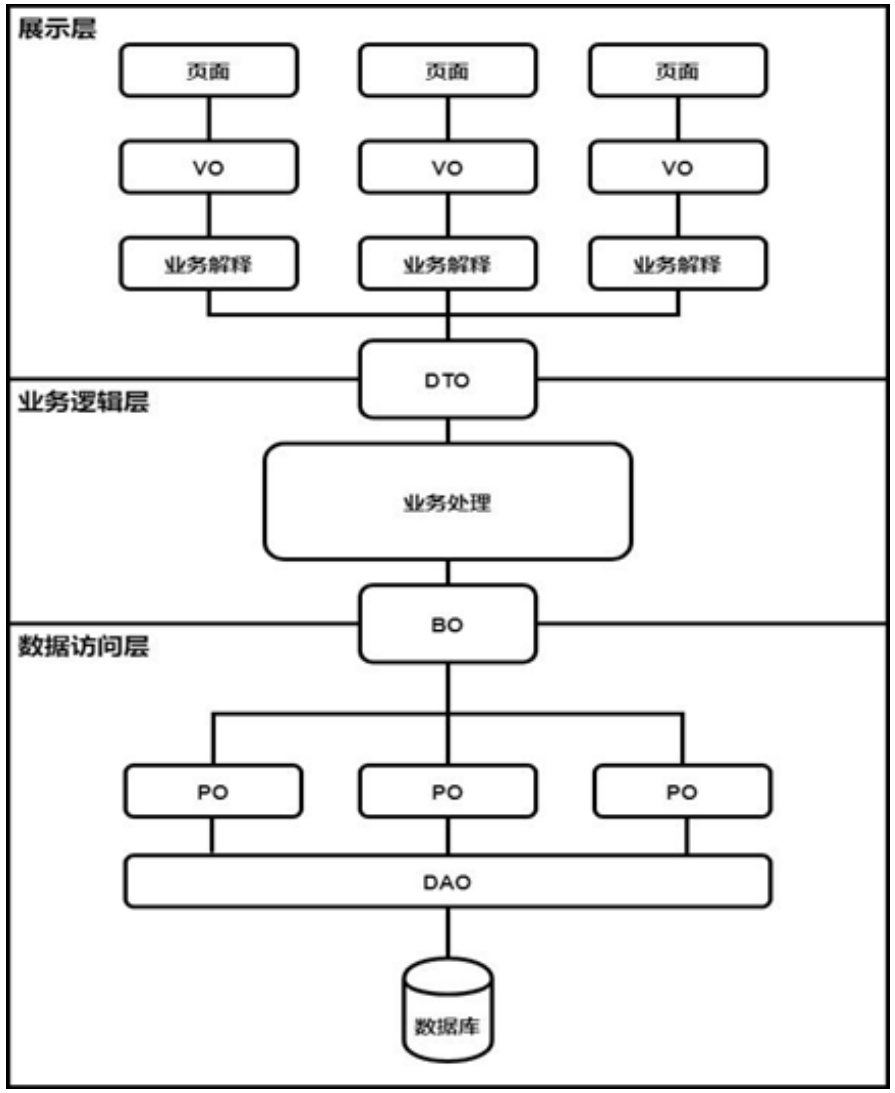
后端各层的数据：
总称：POJO

VO	BO	PO
(View Object)	（Business Object）	（Persistent Object）

各层单独定义的实体类 VO、BO，PO，都是只包含数据，不包含业务逻辑的类，叫作贫血模型（Anemic Domain Model），数据与操作分离，破坏了面向对象的封装特性，面向过程风格。

这种开发模式叫作基于贫血模型的开发模式，也是现在常用的一种 Web 项目开发模式（简单）

1.6 破坏封装典型示例 ③ – 定义数据和方法分离的类



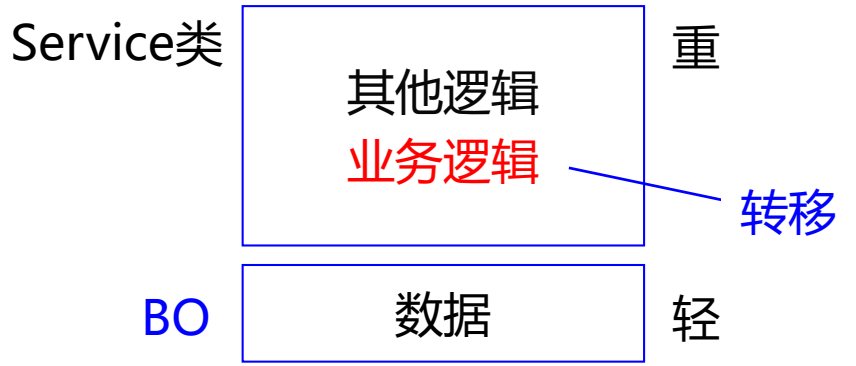
对象	定义	作用	场景	特点
VO	是展示层（前端页面）直接使用的对象	为前端提供所需的数据，避免暴露敏感信息	Controller 层与前端交互时，将数据封装为 VO 返回给前端	仅包含前端需要展示的数据，通常以 JSON 形式返回
DTO	用于不同层之间的数据传输，尤其是 Service 层与 Controller 层的交互	在不同层之间传递数据，减少不必要的数据传输	Service 层与 Controller 层之间的数据传递	可根据需求封装部分字段，支持序列化
BO	封装业务逻辑，可由多个 PO 组合而成	实现复杂的业务规则和操作	业务层，处理业务逻辑	包含复杂业务操作，独立于存储方式
PO	与数据库表结构一一对应，每个字段映射表中的一列	用于数据持久化操作，如增删改查（CRUD）	数据库交互层，与 ORM 框架配合使用	属性与数据库表字段严格对应

1.6 破坏封装典型示例 ③ – 数据和方法分离的类-解决方案

解决方案：数据和对应的业务逻辑被封装到同一个类中。这种类称为：充血模型（Rich Domain Model）。显然充血模型满足面向对象的封装特性，是典型的面向对象编程风格

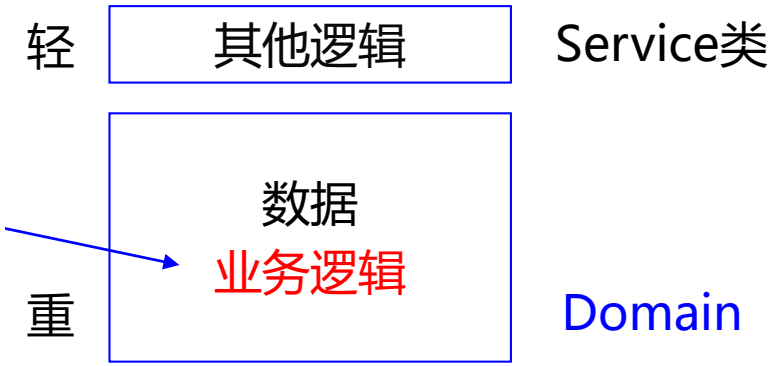


基于贫血模型的传统的开发模式



转移

基于充血模型的 DDD 开发模式



- 基于贫血模型的传统的开发模式，重 Service 轻 BO；
- 基于充血模型的 DDD 开发模式，轻 Service 重 Domain。

1.6 破坏封装典型示例 ③ – 数据和方法分离的类-解决方案

```
public class OrderService {
    private OrderRepository orderRepository; //通过构造函数或者 IOC 框架注入
    public void confirmOrder(Order order) {
        if(order.getItems().isEmpty()){
            throw new IllegalStateException("订单不为空");
        }
        order.setStatus("CONFIRM");
        orderRepository.save(order);
    }
     public void addItem(OrederItem item) {
        if (status != OrderState.DRAFT) {throw new IllegalStateException( “已确认订单不可修改”); }
        items.add(item);
        totalAmount = totalAmount.add(item.calculateTotal());
    }
}
```

基于贫血模型的传统的开发模式示例代码

OrderService + BO

```
public class Order {  //BO类
    private String orderId;
    private List<OrederItem> items;
    private BigDecimal totalAmount;
    private String status;
    // Getter /Setter 方法省略...（无业务逻辑）
}
```

BO中无业务逻辑，只是数据存储的载体

1.6 破坏封装典型示例 ③ – 数据和方法分离的类-解决方案

```
public class Order {  // 领域（Domain）类
    private String orderId;
    private List<OrederItem> items;
    private BigDecimal totalAmount;
    private String status;
    //业务逻辑内聚到聚合根
     public void addItem(OrederItem item) {
        if (status != OrderState.DRAFT) {throw new IllegalStateException("已确认订单不可修改"); }
        items.add(item);
        totalAmount = totalAmount.add(item.calculateTotal());
    }
    public void confirm() {
        if (order.getItems().isEmpty()) { throw new IllegalStateException("订单不为空");   }
        this.status = OrderState.CONFIRMED;
    }
    // getter方法（无setter，通过 addItem和 confirm方法控制状态变更）
     public String getOrderId() {   return orderId;   }
    public List<OrederItem> getItems() {  return items; }
}
```

修改为：充血模型 Service + Domain

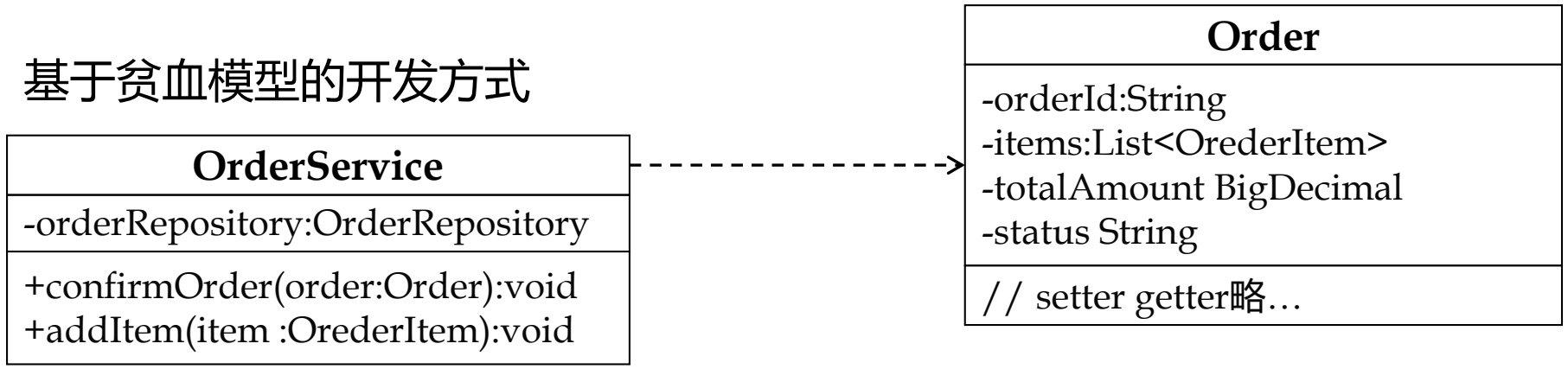
1.6 破坏封装典型示例 ③ – 数据和方法分离的类-解决方案

```
public class OrderService {
    private OrderRepository orderRepository; // 通过构造函数或者 IOC 框架注入
    public void confirmOrder(String orderId) {
        Order order = orderRepository.findById(orderID);
        order.confirm(); //业务逻辑在领域对象中
        orderRepository.save(order);
    }
}
```

基于充血模型的 Service 部分 只做协调工作。领域模型实现自身业务

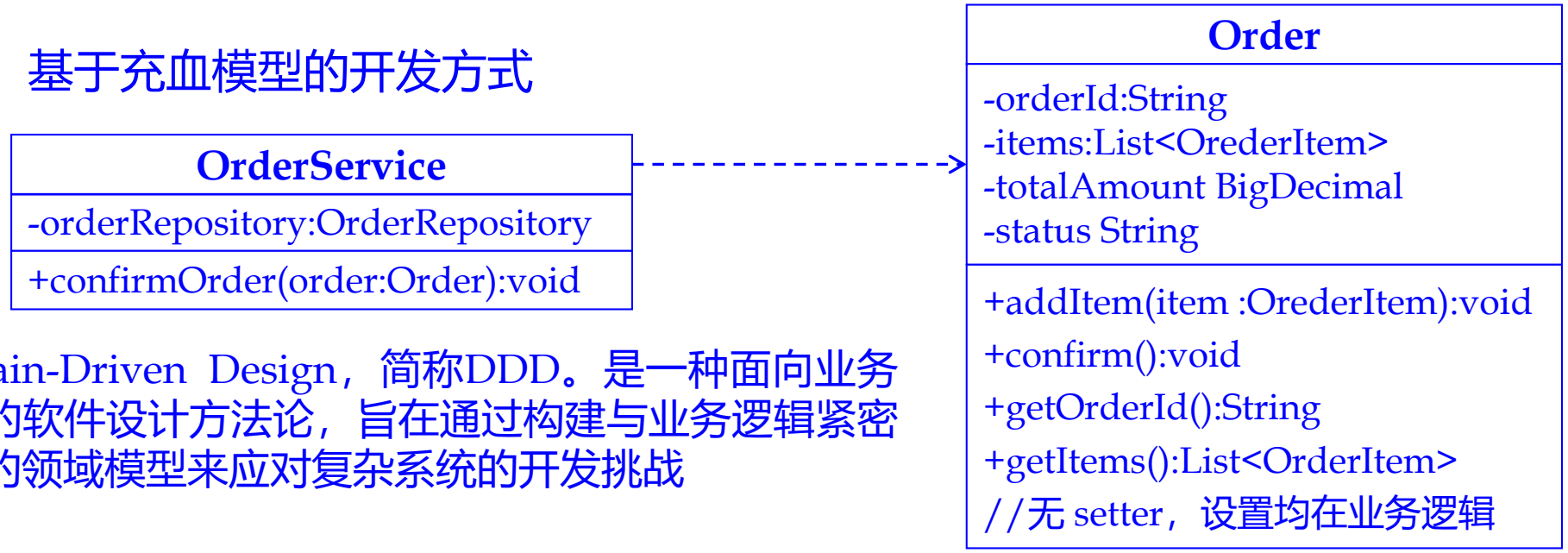
1.6 破坏封装典型示例 ③ – 数据和方法分离的类-小结

基于贫血模型的开发方式



贫血模型

基于充血模型的开发方式



充血模型
(业务领域类)

Domain-Driven Design，简称DDD。是一种面向业务领域的软件设计方法论，旨在通过构建与业务逻辑紧密耦合的领域模型来应对复杂系统的开发挑战

1.6 破坏封装典型示例 ③ – 数据和方法分离的类-小结

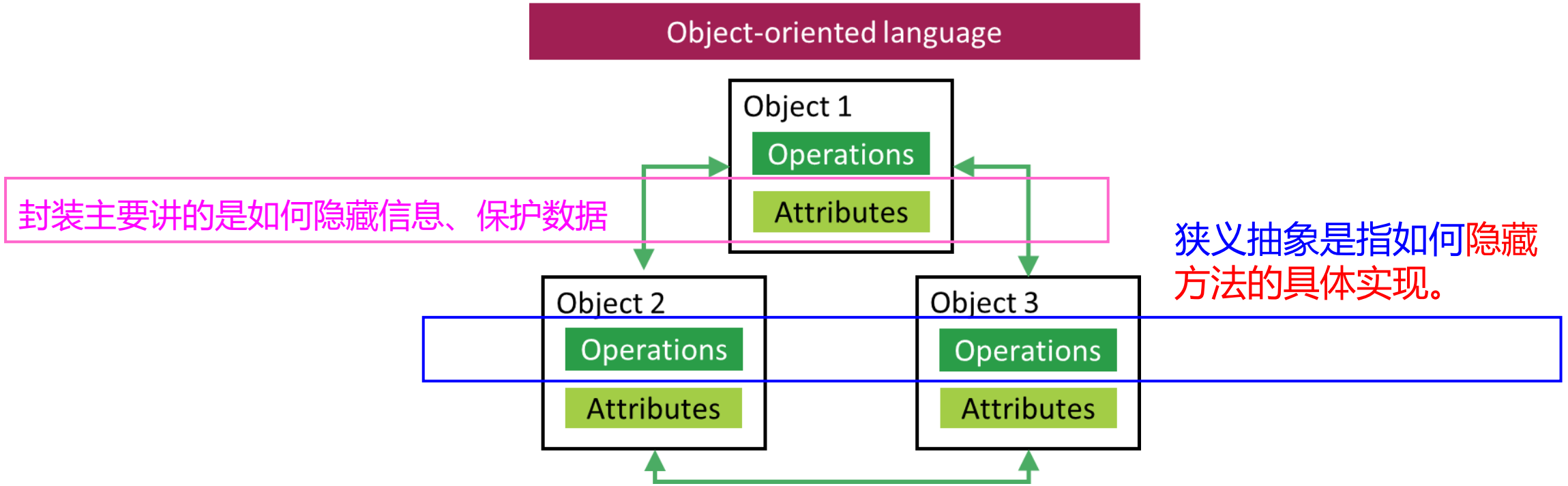
维度	贫血模型	充血模型
业务逻辑位置	分散在 Service类中	内聚在领域对象中
对象职责	数据容器	数据+行为
代码可维护性	低（业务逻辑分散）	低（业务逻辑分散）
面向对象设计	违背封装性	符合封装性
典型应用场景	简单的 CRUD 系统	复杂业务系统（如电商，金融）

2. 抽象
Abstraction

2.1 什么是抽象?

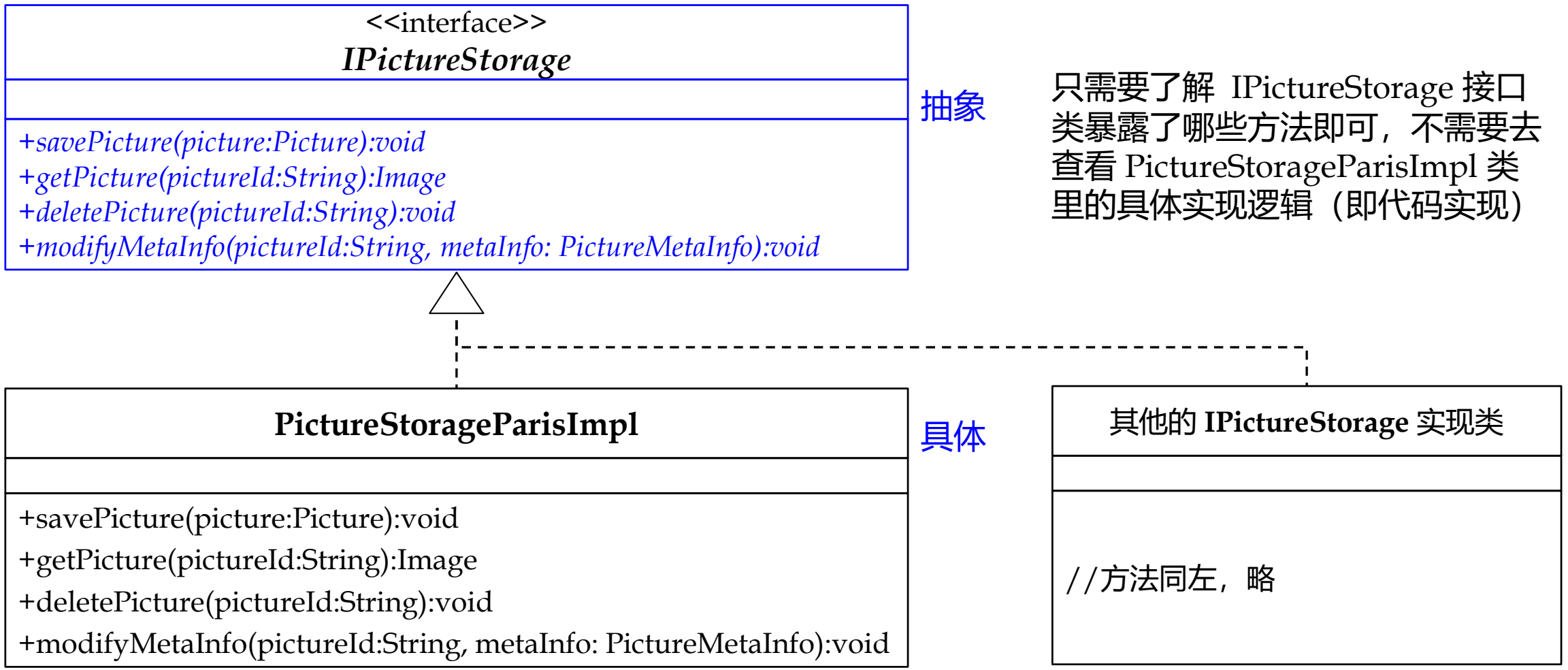
- 广义的抽象概念：抽象（Abstract）就是忽略事物中与当前目标无关的非本质特征，更充分地注意与当前目标有关的本质特征。从而找出事物的共性，并把具有共性的事物划为一类，得到一个抽象的概念。
- 例如：
  - 根据对象抽象出类。
  - 根据具体类抽象出超类。
- 狭义的抽象概念：在面向对象的四大特征方面，可以具体化为：隐藏方法的具体实现。

2.1 什么是抽象?



借助编程语言提供的接口类（如 Java 中的 interface 关键字语法）或者抽象类（如 Java 中的 abstract 关键字语法）这两种语法机制，来实现抽象这一特性。

2.2 抽象的实例说明


<<interface>>
IPictureStorage
+savePicture(picture:Picture):void
+getPicture(pictureId:String):Image
+deletePicture(pictureId:String):void
+modifyMetaInfo(pictureId:String, metaInfo: PictureMetaInfo):void
抽象
只需要了解 IPictureStorage 接口类暴露了哪些方法即可，不需要去查看 PictureStorageParisImpl 类里的具体实现逻辑（即代码实现）
PictureStorageParisImpl
+savePicture(picture:Picture):void
+getPicture(pictureId:String):Image
+deletePicture(pictureId:String):void
+modifyMetaInfo(pictureId:String, metaInfo: PictureMetaInfo):void
具体
其他的 IPictureStorage 实现类
//方法同左，略

2.2 抽象的实例说明

抽象

```
public interface IPictureStorage {
    void savePicture(Picture picture);
    Image getPicture(String pictureId);
    void deletePicture(String pictureId);
    void modifyMetaInfo(String pictureId, PictureMetaInfo metaInfo);
}
```

具体

```
public class PictureStorage implements IPictureStorage {
    // ... 省略其他属性...
    @Override
    public void savePicture(Picture picture) {  //代码略…   }
    @Override
    public Image getPicture(String pictureId) { //代码略…   }
    @Override
    public void deletePicture(String pictureId) { //代码略…  }
    @Override
    public void modifyMetaInfo(String pictureId, PictureMetaInfo metaInfo) { //代码略… }
}
```

2.3 抽象有时候会被排除在面向对象的四大特性之外

```
public class PictureStorage {
    // ... 省略其他属性...

    public void savePicture(Picture picture) { //具体实现代码，略... }

    public Image getPicture(String pictureId) { //具体实现代码，略... }

    public void deletePicture(String pictureId) { //具体实现代码，略... }

    /**
     *
     * 输入、输出、返回值和处理逻辑的文档说明
     */
    public void modifyMetaInfo(String pictureId, PictureMetaInfo metaInfo) {

        // 具体代码，略...
    }
```

即便在普通类中，调用者并不需要去研究方法的内部实现逻辑，只需要了解方法签名就可以调用。

例如 Java api文档

方法签名

因此：抽象特性并不需要编程语言提供特殊的语法机制来支持，只需要提供“函数”这一非常基础的语法机制，就可以实现。所以，它没有很强的“特异性”，有时候并不被看作面向对象编程的特性之一

2.4 抽象解决的编程问题 – 隐藏方法实现，实现信息过滤

抓重点：过滤细节

- 面对复杂系统的时候，人脑能承受的信息复杂程度是有限的，所以我们必须忽略掉一些非关键性的实现细节。
- 抽象作为一种只关注功能点不关注实现的设计思路，正好帮我们的大脑过滤掉许多非必要的信息

抽象作为一个非常宽泛的设计思想，在代码设计中，起到非常重要的指导作用

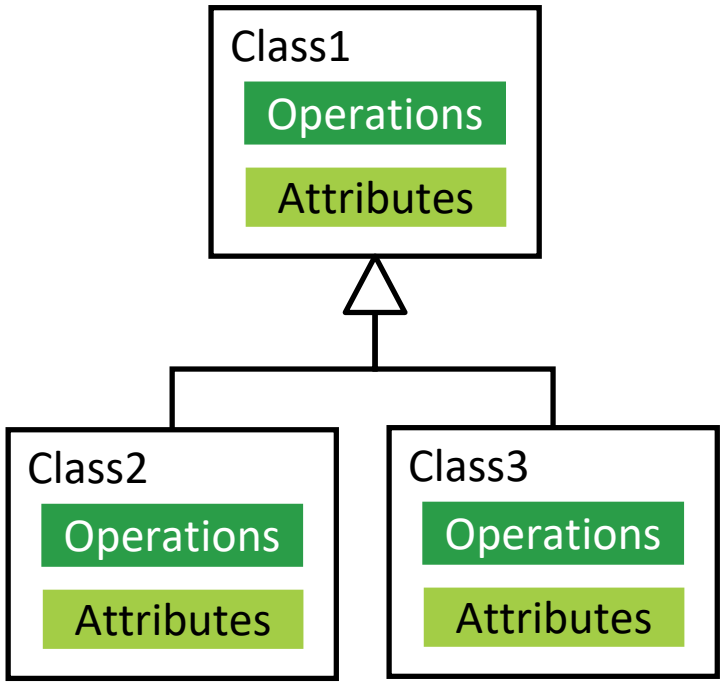
- 很多设计原则都体现了抽象这种设计思想：例如基于接口而非现编程、开闭原则，代码解耦等
- 定义类中方法时，也要有抽象思维，不要暴露实现细节。
  例如：getAliyunPictureUrl() 就不是一个具有抽象思维的命名。当不再把图片存储在阿里云上时，这个命名也要随之被修改。
  具有抽象思维的命名方法：getPictureUrl()

3. 继承

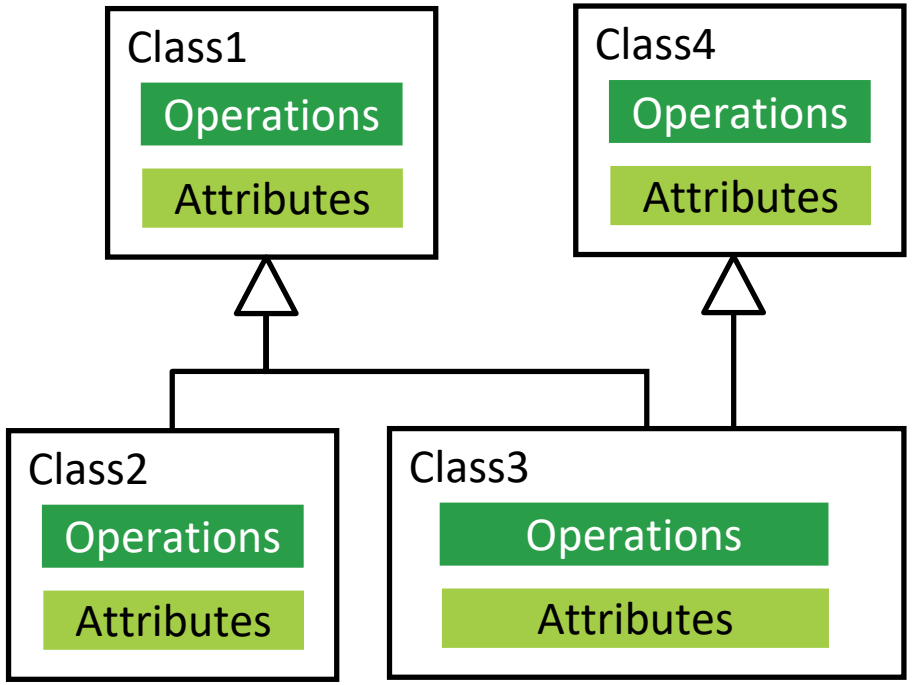
Inheritance，美[ɪnˈherɪtəns]

3.1 继承的特性

- 继承是用来表示类之间的“is-a”关系。



单继承：一个子类只继承一个父类



多继承：一个子类可以继承多个父类。例如Class3
部分语言不允许多继承（例如：Java）

3.2 继承解决的编程问题 – “is a”关系落地，代码复用

解决人类认知的“is-a”问题

如果两个类存在“is-a”的关系，可以通过继承来关联两个类，反应真实世界中的这种关系，非常符合人类的认知

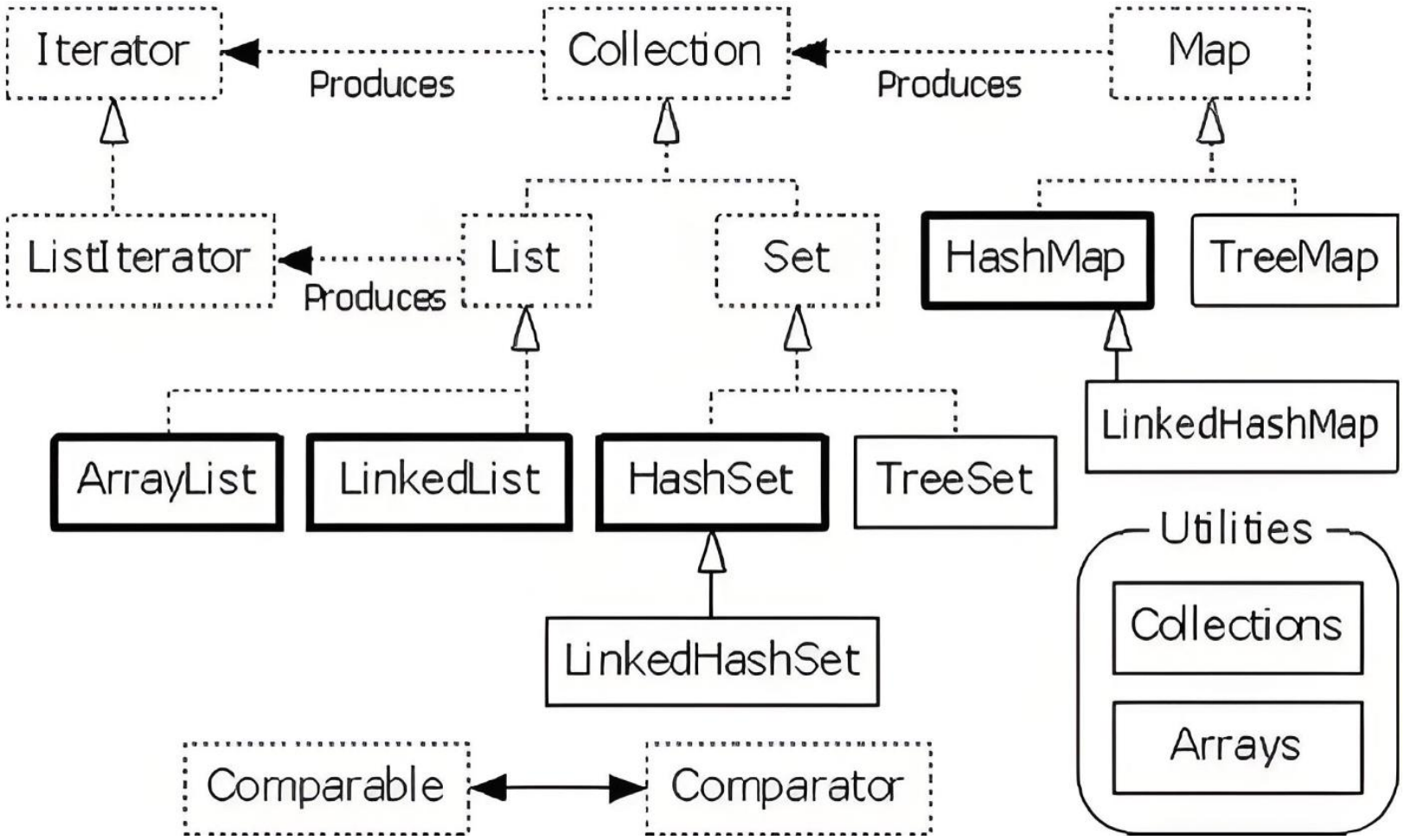
而且，从设计的角度来说，也有一种结构美感。

部分解决代码复用(Reuse)问题

假如两个类有一些相同的属性和方法，就可以将这些相同的部分，抽取到父类中，让两个子类继承父类。

这样两个子类就可以重用父类中的代码，避免代码重复写多遍

3.3 继承特性具有争议性 – 建议多用组合，少用继承



问题1：继承层次过深、过复杂，就会导致代码可读性、可维护性变差。

为了了解一个类的功能，需要查看所有的祖先

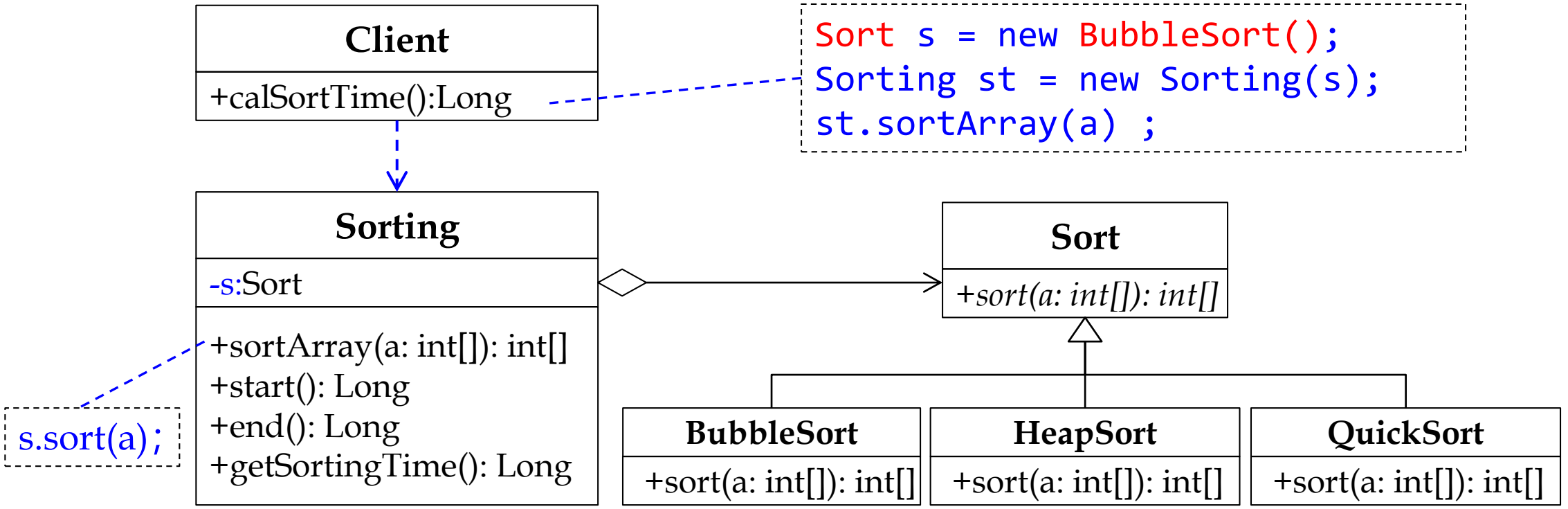
问题2：子类和父类是高度耦合的，因此，修改父类的代码，会直接影响到子类

4. 多态

Polymorphism，美[ˌpɑˌliˈmɔrfɪzm]

4.1 什么是多态

- 多态是指：子类可以替换父类，在实际的代码运行过程中，调用子类的方法实现。



4.2 多态解决的编程问题 – 解决代码冗余，提高扩展性

解决代码冗余问题	多态通过继承和重写方法，使父类引用指向子类对象时自动调用子类方法，避免重复代码；
提高扩展性	新增类型时仅需扩展子类无需修改父类代码，提升系统可维护性； 利用多态去掉冗长的 if-else 语句

本章总结

面向对象特性	概念	解决的编程问题
封装 (Encapsulation)	隐藏数据。类通过暴露有限的访问接口，授权外部访问内部数据	保护数据不被随意修改，提高代码的可维护性；是仅暴露有限的必要接口，提高类的易用性。

本章总结

面向对象特性	概念	解决的编程问题
抽象 (Abstraction)	隐藏方法的具体实现。让使用者只需要关心方法提供了哪些功能，不需要知道这些功能是如何实现的	提高代码的可扩展性、维护性，修改实现不需要改变定义，减少代码的改动范围； 是处理复杂系统的有效手段，能有效地过滤掉不必要关注的信息。

本章总结

面向对象特性	概念	解决的编程问题
继承 (Inheritance)	继承是用来表示类之间的 is-a 关系	主要是用来解决代码复用的问题

本章总结

面向对象特性	概念	解决的编程问题
多态 (Polymorphism)	多态是指子类可以替换父类，在实际的代码运行过程中，调用子类的方法实现	解决代码冗余问题：多态通过继承和重写方法，使父类引用指向子类对象时自动调用子类方法，避免重复代码； 提高可维护性：新增类型时仅需扩展子类无需修改父类代码，利用多态去掉冗长的 if-else 语句

本章总结

面向对象特性	概念	解决的编程问题
封装 (Encapsulation)	隐藏数据。类通过暴露有限的访问接口，授权外部访问内部数据	保护数据不被随意修改，提高代码的可维护性； 是仅暴露有限的必要接口，提高类的易用性。
抽象 (Abstraction)	隐藏方法的具体实现。让使用者只需要关心方法提供了哪些功能，不需要知道这些功能是如何实现的	提高代码的可扩展性、维护性，修改实现不需要改变定义，减少代码的改动范围； 是处理复杂系统的有效手段，能有效地过滤掉不必要关注的信息。
继承 (Inheritance)	继承是用来表示类之间的 is-a 关系	主要是用来解决代码复用的问题
多态 (Polymorphism)	多态是指子类可以替换父类，在实际的代码运行过程中，调用子类的方法实现	解决代码冗余问题：多态通过继承和重写方法，使父类引用指向子类对象时自动调用子类方法，避免重复代码； 提高可维护性：新增类型时仅需扩展子类无需修改父类代码，利用多态去掉冗长的 if-else 语句